



Atenção

- Este relatório não é uma entrega separada: ele deve ser enviado junto com o .tar da atividade, no mesma atividade no Google Classroom.
- O relatório deve ter como nome as iniciais do seu e-mail em minúsculas (ex.: mla@cesar.school → diretório mla). Fora desse padrão, o relatório é descartado automaticamente.
- Cópia identificada, de outro(a) aluno(a) ou de qualquer outra fonte, descarta toda a submissão, código e relatório juntos.

1. Objetivo (3–5 linhas)

Desenvolver o ProcessFlow, um orquestrador de processos capaz de cadastrar e executar tarefas em diferentes modos. O programa contempla execução simples, sequencial, paralela, pipelines, redirecionamentos, execução em background e processamento de workflows. A entrega contém o programa desenvolvido, arquivos necessários para compilação e execução, evidências dos testes realizados e este relatório.

2. Checklist de Requisitos

Um requisito por linha. Sem parágrafo, se precisar explicar algo, use a coluna "como testar".

Requisito do enunciado	Status	Como testar / evidência
Execução simples de tarefa cadastrada	OK	Cadastrar task listar /bin/ls -l e executar run listar.
Comando exit	OK	Digitar exit e verificar o retorno ao terminal.
Execução sequencial	OK	Executar run sequencial com tarefas cadastradas e observar cada término antes da próxima execução.
Execução paralela	OK	Executar run parallel e observar a criação dos processos antes das esperas.
Pipe	OK	Executar run pipe listar ordenar contar e conferir o resultado final.
Redirecionamento input	OK	Configurar input mostrar saída.txt, executar run mostrar e verificar que a tarefa lê o conteúdo do arquivo como entrada.
Redirecionamento output	OK	Configurar output escrever saída.txt, executar run escrever e conferir que a saída foi gravada no arquivo.
Redirecionamento append	OK	Configurar append adicionar saída.txt, executar run adicionar e verificar que a nova saída foi acrescentada sem apagar o conteúdo anterior.
Background + jobs + wait	OK	Executar uma tarefa com start, conferir seu job em jobs e utilizar wait <jobld> para aguardar o processo correspondente.
WorkflowFile	OK	Executar o programa com um arquivo .pf e verificar o processamento das linhas sem o prompt interativo.
Workdir	OK	Executar workdir /tmp e verificar a alteração do diretório de trabalho.

Requisito do enunciado	Status	Como testar / evidência
Tratamento de erros e outras situações	Parcial	Testar falhas de fork, exec, abertura de arquivo e entradas inválidas.

3. Como Reproduzir

Compilar:
make clean
make

Executar no modo interativo:
./processflow

Executar utilizando um arquivo de workflow:
./processflow teste.pf

4. Arquitetura (resumida)

Arquivos e responsabilidades:

1. **main.c** → leitura dos comandos, armazenamento das tarefas e execução dos diferentes modos do ProcessFlow.
2. **Makefile** → automatiza a compilação do main.c, gera o executável processflow e permite limpar o executável com make clean.
3. **README.md** → contém as instruções básicas para compilar, executar e utilizar os comandos do programa.

Decisões técnicas:

1. **Armazenar as tarefas em um vetor de structs Comando** → manter nome, programa e argumentos juntos → facilita localizar uma tarefa cadastrada antes de executá-la.
2. **Utilizar fork(), execvp(), wait() e waitpid()** → executar os programas através de processos filhos sem utilizar system() → permite controlar execução simples, sequencial, paralela e em background.
3. **Utilizar pipe(), dup2() e descritores de arquivos** → controlar a entrada e saída dos processos → permite implementar pipelines e os redirecionamentos input, output e append. Nos redirecionamentos, a configuração do arquivo fica armazenada na própria tarefa e é aplicada quando ela é executada com run.
4. **Armazenar na struct Comando as configurações de input, output e append** → associar cada redirecionamento à tarefa cadastrada → evita executar a tarefa no momento em que o redirecionamento é configurado.

5. Estratégias e Diário de Desenvolvimento

5.1 Estratégias da semana

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	Entender processos pai e filho	Comecei fazendo testes menores com fork(), wait() e execvp() para entender a criação e execução de processos.	Os testes isolados serviram para entender as chamadas de sistema, mas depois foi necessário adaptar essa lógica para os comandos do ProcessFlow.
S2	Cadastro e modos de execução	Passei a interpretar a entrada, armazenar tarefas em structs e reutilizar a execução com processos filhos para run, sequencial e paralelo.	A mudança ocorreu para sair dos testes isolados e implementar os comandos exigidos pelo projeto.

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S3	Comunicação e recursos adicionais	A partir da execução já funcionando, implementei pipe, redirecionamentos, workflow, workdir e espera de processos específicos. Depois dos testes, ajustei os redirecionamentos para que input, output e append configurassem a tarefa e fossem aplicados apenas no momento do run.	Os requisitos restantes exigiam manipular descritores, arquivos e formas diferentes de controle dos processos. Os testes posteriores também mostraram que a lógica inicial dos redirecionamentos precisava ser adaptada ao comportamento esperado.

5.2 Diário de Tentativas

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	S1	Simular a criação de um processo pai e um processo filho com fork()	Sucesso	-	19/08/2026	evidencias.log : sessão de 20/08 com criação dos processos pai e filho e exibição dos respectivos PIDs.
2	S1	Evoluir o teste dos processos adicionando wait() em ambiente Linux	Sucesso	-	19/08/2026	evidencias.log : sessão de 20/08 mostrando o processo pai aguardando o término do processo filho.
3	S1	Executar um programa através do processo filho e tratar erro do exec	Sucesso	-	19/08/2026	evidencias.log : sessão de 20/08 com falha do exec registrada como "No such file or directory".
4	S2	Quebrar a linha de entrada com strtok(), criar a struct Comando e começar a separar task de run	Parcial	A entrada já era interpretada e armazenada, mas a execução da tarefa cadastrada ainda não estava completa.	20/08/2026	evidencias.log : sessão de 20/08 com leitura de task listar /bin/ls -l e separação da entrada em nome, programa e argumentos.
5	S2	Separar definitivamente task e run, armazenar as tarefas e executar a tarefa localizada pelo nome	Sucesso	-	21/08/2026	evidencias.log : cadastro de task listar /bin/ls -l seguido de run listar, localização da tarefa, criação do filho e saída do /bin/ls -l.
6	S2	Implementar a execução de várias tarefas em modo sequencial	Sucesso	-	22/08/2026	evidencias.log : execução de run sequencial um dois tres, com as saídas UM, DOIS e TRES em ordem e o pai aguardando cada filho.
7	S2	Reutilizar a lógica do sequencial para implementar a execução paralela	Sucesso	-	22/08/2026	evidencias.log : execução de run parallel um dois tres, com criação dos processos filhos e término em ordem diferente da execução sequencial.
8	S3	Adaptar a lógica já utilizada nos modos anteriores para criar o pipeline e controlar os descritores	Sucesso	-	23/08/2026	evidencias.log : execução de run pipe listar ordenar contar, com três processos e resultado final 8.

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
9	S3	Implementar os três tipos de redirecionamento: input, output e append	Parcial	A primeira versão executava a tarefa no próprio comando de redirecionamento, em vez de apenas configurar a tarefa.	23/08/2026	evidencias.log : testes de input contar teste.txt com resultado 4, output listar e append listar, seguidos da conferência dos arquivos.
10	S3	Implementar a leitura de comandos por workflow e a alteração do diretório com workdir	Sucesso	-	23/08/2026	evidencias.log : execução de ./processflow teste.pf e teste de workdir /tmp, confirmado posteriormente pela saída /tmp de /bin/pwd.
11	S3	Implementar execução em background com start, listagem com jobs e espera com wait	Sucesso	-	23/08/2026	evidencias.log: teste de start, jobs e wait com tarefa /bin/sleep, mostrando a criação do job e a utilização de wait com o jobId correspondente.
12	S3	Corrigir a lógica dos redirecionamentos após os testes finais	Sucesso	A configuração de input, output e append passou a ser armazenada na tarefa e aplicada apenas quando run executa essa tarefa.	24/08/2026	evidencias.log: teste com output escrever saida.txt, append adicionar saida.txt e input mostrar saida.txt, seguido de run, resultando nas linhas "linha-um" e "linha-dois".

6. Evidências

21/08/2026 : Cadastro e execução simples de tarefas

O evidencias.log registra o cadastro da tarefa listar utilizando /bin/ls -l e sua execução com run listar. O log mostra a localização da tarefa cadastrada, a criação do processo filho, a espera do processo pai e a saída do programa executado.

22/08/2026 : Execução sequencial

Foram cadastradas as tarefas listar, eco e data e executado run sequencial listar eco data. O log mostra que o processo pai aguardou o término de cada filho antes de iniciar a tarefa seguinte.

22/08/2026 : Execução paralela

Foram cadastradas três tarefas e executado o modo run parallel. A evidência mostra a criação dos processos filhos antes da etapa de espera, diferenciando esse comportamento da execução sequencial.

23/08/2026 : Pipeline

Foi executado run pipe listar ordenar contar, utilizando tarefas correspondentes a ls, sort e wc -l. O teste mostrou a comunicação entre os processos e a apresentação do resultado final antes do retorno ao prompt.

23/08/2026 : Redirecionamentos

Foram testados os três tipos de redirecionamento. No input, um arquivo com quatro linhas foi utilizado como entrada da tarefa contar e o resultado foi 4. No output, a saída da tarefa foi gravada em arquivo. No append, novas saídas foram acrescentadas sem apagar o conteúdo anterior.

24/08/2026 : Correção e validação final dos redirecionamentos

Após revisar o comportamento esperado, os comandos input, output e append foram ajustados para configurar o redirecionamento da tarefa, deixando a execução para o comando run. No teste final, output escrever

saida.txt gravou “linha-um”, append adicionar saida.txt acrescentou “linha-dois” e input mostrar saida.txt foi utilizado pela tarefa mostrar. A execução de run mostrar apresentou corretamente as duas linhas do arquivo.

23/08/2026 : Workflow e workdir

Foi testada a execução através de um arquivo .pf, com as linhas sendo apresentadas e processadas sem o prompt interativo. O comando workdir também foi testado alterando o diretório de trabalho e verificando a mudança.

24/08/2026 : Background, jobs e wait

Foi cadastrada uma tarefa utilizando /bin/sleep e executada com start. O programa atribuiu um jobId e exibiu o processo em jobs. Em seguida, wait 1 aguardou o job correspondente e o teste com wait 99 foi tratado como job inexistente sem encerrar o programa.

24/08/2026 : Makefile

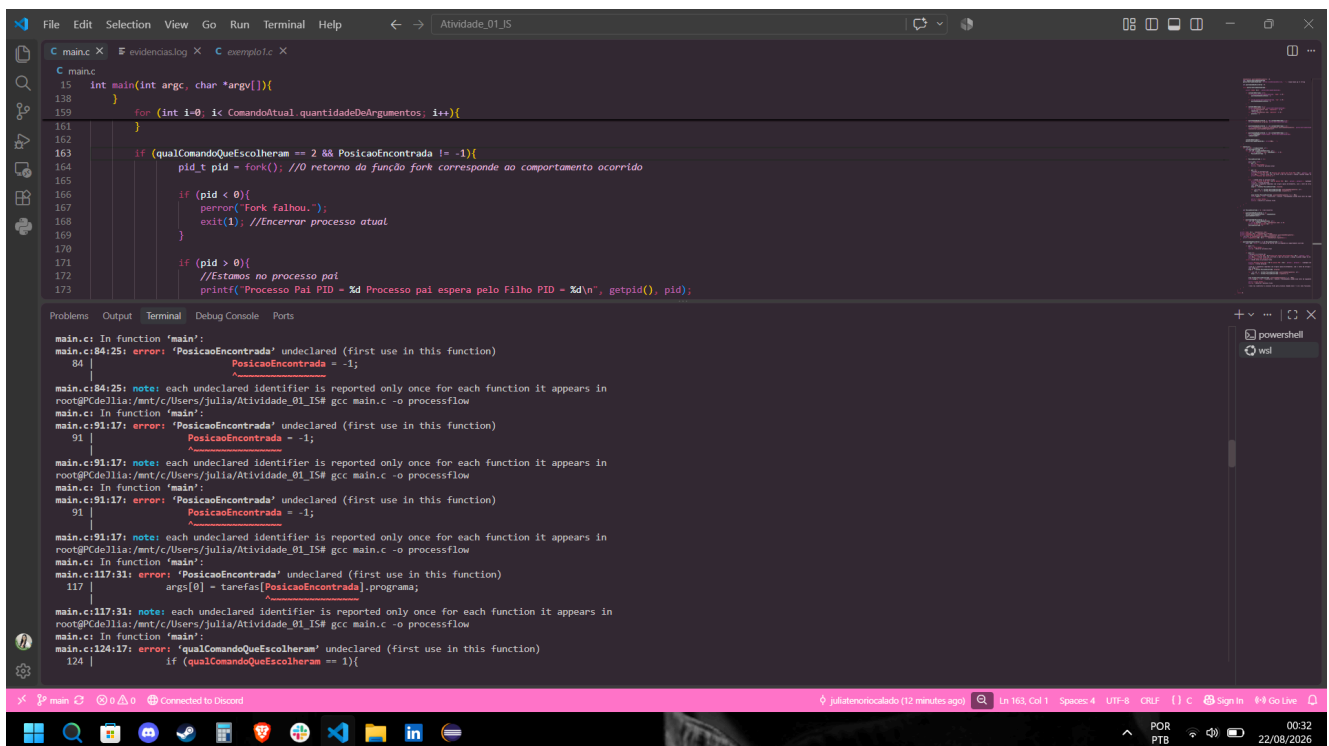
O evidencias.log registra uma compilação do zero utilizando make clean e make. O comando make clean removeu o executável e o make compilou main.c com gcc -Wall -Wextra, gerando novamente processflow.

6.1 Log automático (faz o trabalho pesado por você)

O arquivo evidencias.log foi utilizado durante o desenvolvimento para registrar comandos e saídas do terminal. As sessões foram identificadas com date, whoami e pwd, e o arquivo será incluído no .tar da entrega.

O evidencias.log deve ser enviado dentro do .tar

6.2 Prints



The screenshot shows a VS Code editor with a C program in main.c and its compilation errors in the Output window. The C program is a simple fork-based process flow. The errors in the Output window are as follows:

```
main.c: In function 'main':
main.c:84:25: error: 'PosicaoEncontrada' undeclared (first use in this function)
   84 |         PosicaoEncontrada = -1;
      |         ^
main.c:84:25: note: each undeclared identifier is reported only once for each function it appears in
root@CDEllia:/mnt/c/Users/julia/Atividade_01_IS# gcc main.c -o processflow
main.c: In function 'main':
main.c:91:17: error: 'PosicaoEncontrada' undeclared (first use in this function)
   91 |         PosicaoEncontrada = -1;
      |         ^
main.c:91:17: note: each undeclared identifier is reported only once for each function it appears in
root@CDEllia:/mnt/c/Users/julia/Atividade_01_IS# gcc main.c -o processflow
main.c: In function 'main':
main.c:91:17: error: 'PosicaoEncontrada' undeclared (first use in this function)
   91 |         PosicaoEncontrada = -1;
      |         ^
main.c:91:17: note: each undeclared identifier is reported only once for each function it appears in
root@CDEllia:/mnt/c/Users/julia/Atividade_01_IS# gcc main.c -o processflow
main.c: In function 'main':
main.c:117:31: error: 'PosicaoEncontrada' undeclared (first use in this function)
   117 |         args[0] = tarefas[PosicaoEncontrada].programa;
      |                               ^
main.c:117:31: note: each undeclared identifier is reported only once for each function it appears in
root@CDEllia:/mnt/c/Users/julia/Atividade_01_IS# gcc main.c -o processflow
main.c: In function 'main':
main.c:124:17: error: 'qualComandoQueEscolheram' undeclared (first use in this function)
   124 |         if (qualComandoQueEscolheram == 1){
```

Erro principal: print do erro de compilação envolvendo PosicaoEncontrada, ocorrido durante o desenvolvimento da execução sequencial.

Prompts principais:

- “Qual a diferença entre execução sequencial e paralela usando fork e wait?”
- “Como funciona pipe(fd) e o que representam fd[0] e fd[1]?”
- “Como o dup2 conecta a saída de um processo à entrada de outro?”
- “Como funciona o waitpid para esperar um processo específico?”
- “Me ajude a revisar este trecho e identificar o erro.”
- “Gere um Makefile para compilar main.c e gerar o executável processflow.”

O que validei manualmente e como: Compilei e executei o programa no WSL e testei os comandos diretamente no terminal. Validei manualmente execução simples, sequencial, paralela, pipe, redirecionamentos, workflow, workdir, background, jobs e wait. Também testei o Makefile com make clean e make, confirmando a geração do executável processflow.

8. Reflexão Final (5–8 linhas)

Reflexão: Provavelmente o que eu mudaria, se tivesse mais tempo, seria principalmente a organização do código. Toda a lógica ficou em um arquivo só, o que me ajudou a visualizar o funcionamento durante o desenvolvimento, mas também deixou o código um pouco poluído e repetitivo. Os testes finais também mostraram a importância de conferir não apenas se um recurso funciona, mas se ele funciona exatamente da forma esperada, como aconteceu com os redirecionamentos. Eu separaria partes repetidas em funções menores e organizaria melhor as variáveis e os diferentes modos de execução. Acho que isso facilitaria a manutenção, os testes e a identificação de erros.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	Sim
Rodei os testes e evidencias.log foi gerado	Sim
Tenho prints obrigatórios	Sim
Testei pelo menos um caso limite e um caso inválido	Sim
Preenchi a seção de uso de IA	Sim
Revisei o relatório e removi frases genéricas / vazias	Sim

10. Se eu tivesse mais 2 horas

Se eu tivesse mais duas horas, eu continuaria ampliando os testes com casos inválidos e situações menos comuns, como tarefa inexistente, linha vazia e workflows incompletos. Também revisaria todos os retornos das chamadas de sistema para melhorar o tratamento de erros. Outro ponto seria remover mensagens de depuração que não fossem necessárias na versão final e reduzir a repetição de código, separando trechos em funções menores. Por fim, faria uma nova rodada completa de testes usando apenas o README e o relatório como guia, para conferir se outra pessoa conseguiria reproduzir o funcionamento do programa sem depender do histórico de desenvolvimento.

Regras de Integridade (não mudam)

- Ocorrência relevante sem evidência (log ou print) = não conta para a nota.
- Contradição entre prints, log e relato = relatório perde credibilidade.
- Indício de evidência fabricada = entrega invalidada.